THREADED BST PREDECESSOR AND SUCCESSOR

Problem Statement:

Develop a C++ program to find and display the inorder predecessor and successor of a specified node in a threaded binary search tree (BST). The program should build the threaded BST by inserting nodes, replace null child pointers with threads to inorder neighbours, and locate the given node. It should then display its predecessor and successor, wherever they exist, without using recursion or a stack.

Objective:

The objective of this project is to implement a threaded binary search tree in C++ and efficiently determine the inorder predecessor and successor of any specified node. It aims to demonstrate how threads replace null pointers to enable fast traversal without recursion or stack usage. The project also strengthens understanding of BST insertion, searching and inorder traversal, while handling edge cases where a predecessor or successor does not exist.

Algorithm:

Step 1: Start the program and set root = NULL.

Step 2: Define a node with data, left, right, lthread and rthread. A new node has both flags true and both pointers NULL.

Step 3: Read a key to insert. If the tree is empty, make the new node the root.

Step 4: Otherwise, start at the root and move left if key < data, or right if key > data. Stop when the next link is a thread, and ignore duplicate keys.

Step 5: If the key is smaller than the parent, set new->left = parent->left and new->right = parent. Then set parent->left = new and parent->lthread = false.

Step 6: If the key is larger than the parent, set new->right = parent->right and new->left = parent. Then set parent->right = new and parent->rthread = false.

Step 7: Read the key whose predecessor and successor are required, and search for it by following only real child links.

Step 8: If the node is not found, display "Node not found" and stop.

Step 9: For the successor, if rthread = true, the successor is node->right, which may be NULL.

Step 10: If rthread = false, go to the right child and keep moving left until lthread = true. That node is the successor.

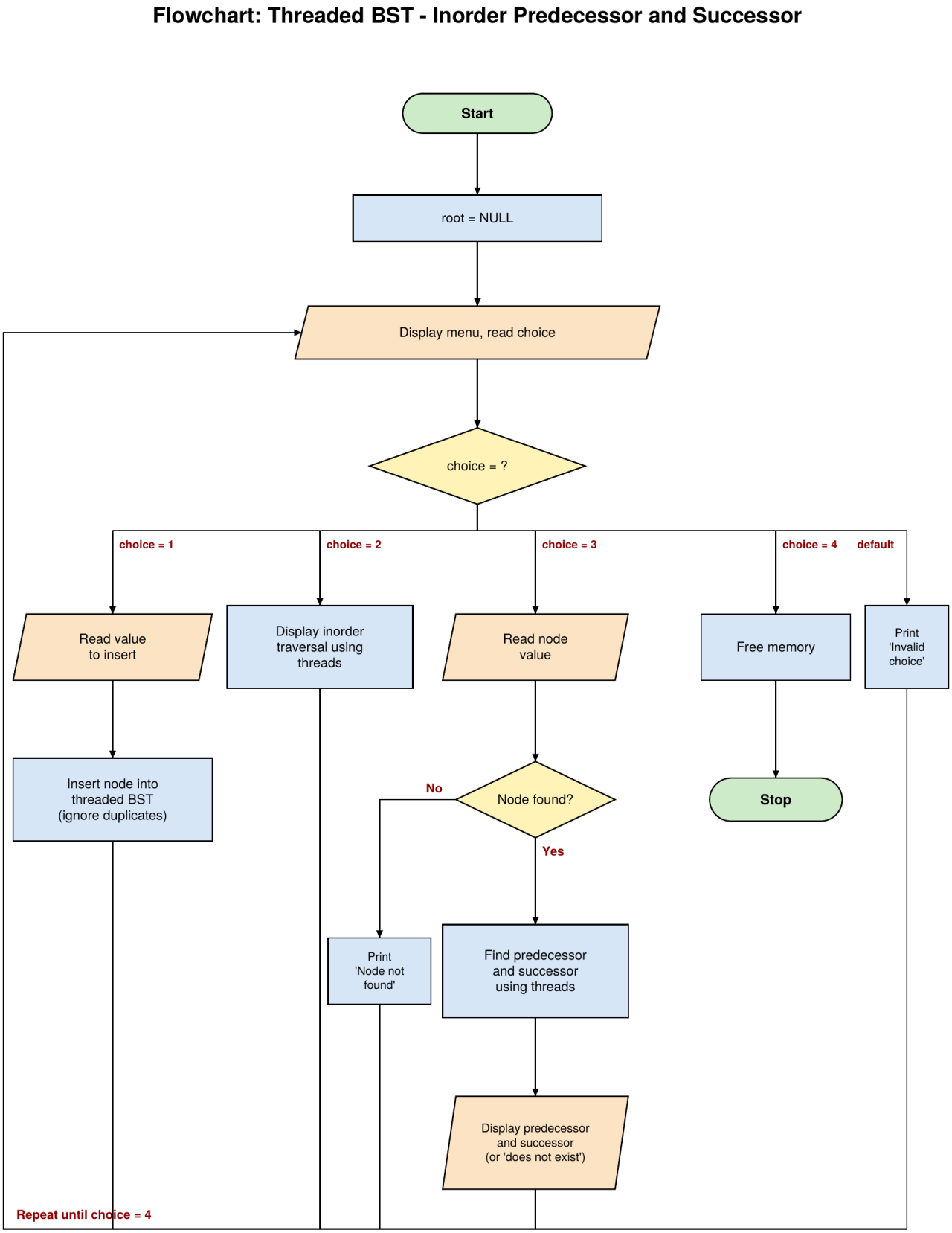
Step 11: For the predecessor, if lthread = true, the predecessor is node->left, which may be NULL.

Step 12: If lthread = false, go to the left child and keep moving right until rthread = true. That node is the predecessor.

Step 13: Display the predecessor and successor, or print "does not exist" where the value is NULL.

Step 14: Free the allocated memory and stop the program.

Flowchart:



Program: threaded_bst.cpp (menu-driven, double-threaded BST)